

1. Specification of Class (Class Definition)

A **class** is a user-defined data type that groups **data members (variables)** and **member functions (methods)** together.

Syntax

```
class ClassName {  
    // data members  
    // member functions  
};
```

Example

```
class Student {  
    int roll;  
    float marks;  
};
```

2. Abstract Data Type (ADT)

An **ADT** defines **what operations are to be performed**, not **how they are implemented**.

In C++, **class acts as an ADT**.

Example

```
class Stack {  
public:  
    void push(int);  
    void pop();  
};
```

Here, the user knows **push()** and **pop()**, but not the internal implementation.

3. Visibility Modes (Access Specifiers)

Visibility modes control **access to class members**.

Specifier	Accessible Inside	Class Outside	Class Derived Class
private	Yes	No	No
protected	Yes	No	Yes
public	Yes	Yes	Yes

Example

```
class Demo {  
private:  
    int a;  
protected:  
    int b;  
public:  
    int c;  
};
```

4. Defining Member Functions

Member functions can be defined:

(a) Inside the Class

```
class Test {  
public:  
    void show() {  
        cout << "Inside class function";  
    }  
};
```

(b) Outside the Class (using Scope Resolution Operator)

```
class Test {  
public:  
    void show();  
};  
  
void Test::show() {  
    cout << "Outside class function";  
}
```

5. Object Creation

An **object** is an instance of a class.

Syntax

```
ClassName objectName;
```

Example

```
Student s1;
```

6. Characteristics of Object

An object has:

1. **Identity** – Unique name (object name)
2. **State** – Data values stored
3. **Behavior** – Functions it can perform

Example

```
Student s1;  
s1.roll = 101;    // state  
s1.display();    // behavior
```

7. Scope Resolution Operator (::)

Used to:

- Define member functions outside the class
- Access static members

Example

```
void Student::display() {  
    cout << roll;  
}
```

8. Static Data Member

- Shared among **all objects**
- Memory allocated **only once**

Declaration

```
static int count;
```

Definition (outside class)

```
int Student::count = 0;
```

9. Static Member Function

- Can access **only static data members**
- Called using class name

Syntax

```
static void showCount();
```

10. Complete C++ Program (Combined Example)

```
#include <iostream>
using namespace std;

class Student {
private:
    int roll;
    float marks;

public:
    static int count;    // static data member

    Student() {
        count++;
    }

    void setData(int r, float m) {
        roll = r;
        marks = m;
    }

    void display();      // member function declaration

    static void showCount() {    // static member function
        cout << "Total Students: " << count << endl;
    }
};

// static data member definition
int Student::count = 0;

// member function definition outside class
void Student::display() {
    cout << "Roll: " << roll << " Marks: " << marks << endl;
}

int main() {
    Student s1, s2;

    s1.setData(101, 85.5);
    s2.setData(102, 90.0);

    s1.display();
    s2.display();

    Student::showCount();    // static function call

    return 0;
}
```

11. Output

```
Roll: 101 Marks: 85.5
Roll: 102 Marks: 90
Total Students: 2
```

In C++, **specification of a class** means **declaring a class by defining its data members and member functions**. It describes **what data the class holds and what operations can be performed on that data**, without necessarily providing implementation details.

General Syntax

```
class ClassName {  
    access_specifier:  
        data_members;  
        member_function_declarations;  
};
```

Key Components of Class Specification

1. **Class keyword**
Used to declare a class.
 2. **Class name**
Identifier that represents the class.
 3. **Access specifiers**
 - o private (default)
 - o protected
 - o public
 4. **Data members**
Variables that store data of the class.
 5. **Member functions**
Functions that operate on the data members.
-

Example: Specification of a Class

```
class Student {  
private:  
    int rollNo;  
    float marks;  
  
public:  
    void setData(int r, float m);  
    void display();  
};
```

◆ In this example:

- Student is the **class name**
 - rollNo and marks are **private data members**
 - setData() and display() are **public member functions**
 - Only **function declarations** are given → this is **class specification**
-

Characteristics of Class Specification

- Defines a **user-defined data type**
 - Acts as a **blueprint for objects**
 - Does **not allocate memory** for data members
 - Specifies **structure and behavior** of objects
 - Supports **data abstraction and encapsulation**
-

Difference between Class Specification and Class Implementation

Class Specification	Class Implementation
Declares members	Defines functions
No memory allocation	Code logic provided
Describes interface	Describes working

Implementation Example (Outside Class)

```
void Student::setData(int r, float m) {  
    rollNo = r;  
    marks = m;  
}  
  
void Student::display() {  
    cout << rollNo << " " << marks;  
}
```

One-Line Exam Definition

Class specification in C++ is the declaration of a class that defines its data members, member functions, and access specifiers, serving as a blueprint for creating objects.

An **Abstract Data Type (ADT)** is a data type that **defines a set of values and the operations that can be performed on those values**, without specifying **how the operations**

are implemented.

It focuses on **what to do**, not **how to do it**.

Key Idea of ADT

- **Data + Operations = ADT**
 - Implementation details are **hidden**
 - Supports **data abstraction**
-

Abstract Data Type in C++

In C++, an **ADT is implemented using a class**.

The **class interface (public members)** represents the ADT, while the **private members** hide the implementation.

Example: Stack as an ADT

ADT Definition (Conceptual)

Operations:

- push()
- pop()
- isEmpty()

User does not know how the stack is implemented (array or linked list).

ADT Implementation in C++ Using Class

```
#include <iostream>
using namespace std;

class Stack {
private:
    int arr[5];
    int top;

public:
    Stack() {
        top = -1;
    }

    void push(int x) {
        if (top == 4)
```

```
        cout << "Stack Overflow\n";
    else
        arr[++top] = x;
}

void pop() {
    if (top == -1)
        cout << "Stack Underflow\n";
    else
        cout << "Popped element: " << arr[top--] << endl;
}
};
```

Usage of ADT (Object Creation)

```
int main() {
    Stack s;
    s.push(10);
    s.push(20);
    s.pop();
    return 0;
}
```

- ✓ The user only uses `push()` and `pop()`
 - ✓ Internal details (`arr`, `top`) are hidden
 - This is **Abstract Data Type behavior**
-

Characteristics of ADT

1. Implementation independent
 2. Provides data abstraction
 3. Enhances security
 4. Easy to modify and maintain
 5. Improves code readability
-

Advantages of ADT

- Reduces program complexity
 - Prevents misuse of data
 - Encourages modular programming
 - Supports reusability
-

ADT vs Data Structure

ADT

Data Structure

Logical description Physical implementation

What operations How operations

User view Programmer view

Common ADTs in C++

- Stack
 - Queue
 - List
 - Set
 - Map
 - Vector (STL)
-

Exam-Oriented One-Line Answer

An Abstract Data Type (ADT) in C++ is a user-defined data type that specifies the data and operations on it, while hiding the implementation details, usually implemented using classes.

In C++, **visibility modes** (also called **access specifiers**) control **how class members are accessed** from different parts of a program. They help achieve **data hiding and encapsulation**.

Types of Visibility Modes in C++

1. `private`

- **Default access specifier** in a class
- Members are accessible **only inside the class**
- Not accessible in derived classes or outside

Example

```
class Demo {  
private:  
    int a;  
};
```

2. `protected`

- Members are accessible:
 - Inside the class
 - In derived (child) classes
- Not accessible outside the class

Example

```
class Demo {
protected:
    int b;
};
```

3. public

- Members are accessible:
 - Inside the class
 - Outside the class
 - In derived classes

Example

```
class Demo {
public:
    int c;
};
```

Example Program Demonstrating Visibility Modes

```
#include <iostream>
using namespace std;

class Sample {
private:
    int x = 10;

protected:
    int y = 20;

public:
    int z = 30;

    void show() {
        cout << x << " " << y << " " << z << endl;
    }
};

int main() {
    Sample obj;
    obj.show();          // allowed
    // obj.x = 5;         // error: private
    // obj.y = 6;         // error: protected
    obj.z = 7;           // allowed
    return 0;
}
```

Accessibility Summary Table

Access Specifier Inside Class Derived Class Outside Class

private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

Importance of Visibility Modes

- Protects data from unauthorized access
 - Supports encapsulation
 - Improves program security
 - Controls interface exposure
-

Exam-Oriented Definition

Visibility modes in C++ are access specifiers (private, protected, public) that control the accessibility of class members to ensure data hiding and encapsulation.

Defining Member Functions Inside the Class

- Function body is written **within the class definition**
- Functions are **implicitly inline**
- Suitable for small functions

Example

```
#include <iostream>
using namespace std;

class Sample {
private:
    int a;

public:
    void set(int x) {
        a = x;
    }

    void show() {
        cout << "Value of a = " << a;
    }
};
```

2. Defining Member Functions Outside the Class

- Function is declared inside the class
- Defined outside using **scope resolution operator (::)**
- Improves readability for large programs

Syntax

```
return_type ClassName::functionName(parameters) {  
    // function body  
}
```

Example

```
#include <iostream>  
using namespace std;  
  
class Sample {  
private:  
    int a;  
  
public:  
    void set(int x);  
    void show();  
};  
  
// Member function definitions  
void Sample::set(int x) {  
    a = x;  
}  
  
void Sample::show() {  
    cout << "Value of a = " << a;  
}
```

Object Creation and Function Call

```
int main() {  
    Sample obj;  
    obj.set(10);  
    obj.show();  
    return 0;  
}
```

Key Points

- Member functions can access **private, protected, and public members**
- **::** helps associate the function with its class
- Inside-class functions are automatically inline
- Outside-class functions improve modularity

Comparison Table

Inside Class	Outside Class
Simple & short	Better for large code
Inline by default	Not inline by default
Less readable for large programs	More organized

Exam-Oriented Concept

Defining member functions in C++ refers to implementing the functions of a class either inside the class definition or outside the class using the scope resolution operator.